

DriftTrust Audit Code and Live Lab Guide

এই guide-এর লক্ষ্য: repository বুঝতে পারা, browser থেকে server পর্যন্ত একটি request অনুসরণ করা, এবং একজন visitor-কে সংভাবে system দেখানো। The deployed product scores measured network flows and studies model adaptation with audit evidence. It does not automatically inspect the visitor's network or grant enterprise access.

Source snapshot 32e224d901843acc4761f25fd02d93f73a5174f0, inspected 9 September 2026. This guide explains that implementation; later changes may alter behavior.

Read the visitor walkthrough first. Then follow the nine modules from architecture to evidence. The companion fieldbook contains interactive exercises; the published lab is a separate networked service.

Visitor walkthrough

1. Open the live lab. Wait for 'Model online · database connected'. Choose 'Audit-only · record policy flags' and click 'Start lab session'. A new isolated model session starts from the published checkpoint.
2. In 'Single flow', choose the actual MQTT_Publish example and click 'Run actual inference'. Look at the trust score, network-risk flag, model fingerprint and contributions. The example's known label is displayed for comparison but is not sent into the inference call.
3. Try DOS_SYN_Hping, then ARP_poisoning. The initial model correctly flags the supplied SYN example and misses the supplied ARP example. A reassuring score can be wrong. Each family has one supplied example, not a guarantee about all its flows.
4. Open 'Dataset replay' and click 'Process next 128 flows'. One browser request is handled as four internal batches of 32. Watch observed-flow and candidate counters. A button click supplies evidence; it does not command the model to adapt.
5. Click 'Run remaining replay'. 'Pause' stops after the current request finishes. There are 1,280 held-out measured flows in a constructed known / novel / known / novel order. The chart shows up to the latest 512 received flow scores.
6. Open the actual candidate records, inspect trigger, ECI, failed checks and model hashes, then click 'Export AJRs'. The prior hosted audit-only verification produced five accepted candidates. Counts describe that clean replay, not arbitrary uploaded inputs.
7. To compare the gate, select 'Gated · reject failed candidates' and click 'Start lab session' again. Merely changing the dropdown does not change the active session policy. Replay from the initial checkpoint; compare decisions and flags. Export the old session first because the new session replaces the browser's session cookie.
8. For your own data, use 'CSV stream' → 'Download CSV example'. Preserve all 12 column names and compatible measurement units. Replace rows with measured flows; optional label is 0 benign, 1 attack, blank unknown. Then choose the file and 'Process CSV stream'. A nontechnical visitor can use supplied examples and replay without collecting traffic.

Live lab: <https://drifttrust-audit-lab.pipratools.chatgpt.site/lab.html>

1 What is actually running

Three connected parts: evidence website, live API, and reproducible experiment.

গল্প and motivation

গল্প / imagined scenario: একজন reviewer শুধু একটি সুন্দর dashboard দেখে জানতে চান—numbers কি button-এর scripted response, নাকি model-এর calculation? এই repository-তে সেই প্রশ্নের উত্তর code path অনুসরণ করে দেওয়া যায়।

সহজ Analogy

সহজ Analogy: index.html হলো museum-এর exhibition, lab.html হলো working laboratory, আর research scripts হলো experiment notebook. Exhibition results দেখায়; laboratory নতুন input হিসাব করে; notebook বলে results কীভাবে তৈরি হয়েছে।

Formal definition and code flow

The current public-data path is research/prepare.py → exported JSON model and streams → runtime/engine.mjs → research/run.mjs → research/report.py. The same engine is imported by runtime/worker.mjs for live requests. Offline evaluation and hosting share equations and update rules, while their stream sizes and runtime timing differ.

index.html + site.mjs + site.css render published results. lab.html defines the controls; lab.mjs validates browser inputs, calls the API and renders returned values. The browser does not train the live session model. The Worker does inference, detection, training, policy evaluation and persistence; D1 holds session state and AJRs.

The legacy Python path remains: main.py coordinates data/simulate.py and data/features.py, models/, drift/, governance/ and evaluation/. It is a synthetic telemetry prototype. Its TemporalMLPTrustScorer flattens a time window into an MLP; its name does not make it an LSTM. The live backend is a separate per-flow 12–24–1 MLP. Legacy label 1 means legitimate; current label 1 means attack. Do not mix their data or report their metrics as one experiment.

Math and worked application

9. Choose 'show paper results': index.html loads assets/research/results.json through site.mjs. No live model update is required.
10. Choose 'score this flow': lab.mjs sends POST /api/infer. worker.mjs loads this session's weights and calls predict() and explain() from engine.mjs.
11. Choose 'regenerate paper': prepare.py, run.mjs and report.py execute the offline protocol; changing the browser display does not regenerate experiments.

Interactive check

Before selecting an action, predict whether it needs only static files, a Worker request, or the offline experiment.

Select a task. The highlighted code path changes between static presentation, live Worker processing and offline experiment. Reset to paper results; no network calls are made.

Recall question: Which file supplies the same model equations to experiments and the live API?

Answer: runtime/engine.mjs engine.mjs is imported by both run.mjs and worker.mjs. main.py is the separate historical Python path.

Explain back: Explain the difference between the evidence page, live lab and old Python prototype.

Self-check: Mention static results versus newly computed predictions. Identify the shared engine and D1 persistence. State that the legacy and current label conventions differ.

Source files: README.md, main.py, models/trust_scorer.py, research/run.mjs, runtime/worker.mjs

2 A visitor's first ten minutes

Start with actual examples; use replay to see adaptation; upload measurements only when available.

গল্প and motivation

গল্প / imagined scenario: একজন visitor-এর কাছে CSV নেই এবং packet-rate মানে কী তাও জানেন না। তাঁকে শুরুতেই 12টি number বানাতে বলা উচিত নয়। Supplied held-out examples and replay give them a valid first route through the existing lab.

সহজ Analogy

সহজ Analogy: Single flow হলো একটি exam answer check করা; replay হলো একের পর এক answer দেখে শিক্ষক কোথায় ভুল করছেন তা বোঝা; labelled CSV হলো visitor-এর নিজের answer sheet, যেখানে correction থাকলে learning সম্ভব।

Formal definition and code flow

Start a session after selecting a policy. It is anonymous, cookie-controlled and isolated from other visitors. No account sign-up is needed. Audit-only accepts trained candidates while recording failed checks. Gated mode rejects a candidate if a required check fails. Neither mode gives a network permission decision.

Single-flow inference reads the current model and returns a score and explanations. It does not increment observed-flow counters, change model weights, or produce an adaptation record. Editing a supplied example creates a hypothetical input; the displayed source label no longer validates the edited values.

Dataset replay supplies actual held-out records and reveals their labels after each 32-flow prediction batch. CSV stream uses visitor inputs. Unlabelled rows are scored and counted but do not provide training feedback. A batch without any labels cannot launch an update, even if its attributions change. Switching policy requires a new session, which also resets the replay cursor and model.

Math and worked application

12. Clean initial session: MQTT_Publish row 3502 gives attack score about 0.000038911; displayed trust rounds to 100.0/100. It is benign, but rounding does not imply zero risk.
13. Clean initial session: DOS_SYN_Hping row 21068 gives attack score about 0.999279; displayed trust is 0.1/100. The network-risk flag agrees with its attack label.
14. Clean initial session: ARP_poisoning row 13793 gives attack score about 0.195059 and trust 80.5/100. This is a false negative. These are locally recomputed checkpoint outputs, not invented teaching results.
15. After inference the stream counter is still zero. After a 128-flow replay request it becomes 128. Candidate count can remain zero because evidence and spacing conditions must pass.

Interactive check

Choose a visitor goal and whether trusted labels are available; predict which control to use and whether it can learn.

Choose example, replay or own CSV and toggle label availability. The recommended controls and learning eligibility update. Reset to first-time visitor; do not imply the offline guide contacts the live service.

Recall question: Why might 'Run actual inference' leave all adaptation counters at zero?

Answer: This endpoint scores one input without learning /api/infer is read-only with respect to model state. Replay or labelled stream processing supplies adaptation evidence.

Explain back: Guide a visitor from first opening the lab to exporting their first audit record.

Self-check: Start a session and use a real supplied example. Run replay and distinguish observed flows from candidates. Export records and reset before comparing policies.

Source files: lab.html, lab.mjs, runtime/worker.mjs, assets/research/examples.json

3 Data provenance and initial training

Real measurements support a bounded experiment, not every proposed enterprise claim.

গল্প and motivation

গল্প / actual project history: the public-data implementation adds UCI RT-IoT2022 to a project that previously used synthetic telemetry. It solves the need for measured flow inputs without pretending those flows contain identity, MFA or access-permission labels.

সহজ Analogy

সহজ Analogy: একটি বইয়ের identical photocopies train এবং test দুই জায়গায় থাকলে exam সহজ দেখাবে।

Deduplication repeated input vectors সরায়; train / validation / test separation আলাদা কাজের জন্য আলাদা বই রাখে। Similar examples can still remain related.

Formal definition and code flow

prepare.py downloads the official archive, verifies its pinned SHA-256, assigns source-row IDs, selects 12 numeric features and maps benign family names to 0 and attacks to 1. It excludes invalid values, removes input-vector groups with conflicting binary labels, and deduplicates selected-feature vectors before splitting. Grouping uses pandas feature hashes; the study does not establish independence of related captures.

Observed counts: 123,117 raw rows, zero invalid selected numeric rows, 2,098 rows in nine conflicting-label groups removed, then 17,701 unique vectors retained. The 103,318 further removals are duplicates after conflict exclusion. Novel attack families ARP_poisoning, DDOS_Slowloris and NMAP_XMAS_TREE_SCAN are exclusively test data. Known-family partitions are 7,297 train, 2,432 validation and 2,433 test; 5,539 novel records join the shifted test, producing 7,972 records.

Only training data determines log1p means and standard deviations. Initial MLP fitting uses balanced training resampling and sklearn Adam; the best checkpoint is chosen by known-family validation log loss, up to 45 epochs with patience seven. Logistic regression and random forest are additional baselines. Validation

supplies fixed canonical anchors; training supplies the initial replay buffer. Five fixed seeds are 11, 23, 37, 53 and 71; seed 11 is the predetermined published model.

No capture timestamps are released in this CSV. Known / novel replay phases are deliberately constructed. Novel means absent from initial train/validation, not an unknown zero-day designation. Labels are assumed available promptly during offline replay. RT-IoT2022 is a captured IoT testbed with generated attacks, not an enterprise access-control field trial.

Math and worked application

16. $123,117 - 2,098 = 121,019$ rows after conflict removal.
17. $121,019 - 103,318 = 17,701$ retained vectors.
18. $7,297 + 2,432 + 2,433 = 12,162$ known-family vectors.
19. $12,162 + 5,539 = 17,701$ total; $2,433 + 5,539 = 7,972$ shifted-test flows.

Interactive check

Predict which partition may supply scaler statistics, anchors and initial weights; assign each source before checking leakage.

Assign sources for scaler, anchors and initial weights. Report protocol leakage or valid selection. Reset to an intentionally incorrect assignment; use actual fixed partition counts.

Recall question: May the initial scaler use all test flows because their labels are hidden?

Answer: No, this protocol fits preprocessing on training only Using the held-out distribution would change the stated protocol. Here means, scales and initial fitting use training; validation selects checkpoints and anchors.

Explain back: Explain what makes this a real-data experiment and why it is not natural chronological drift evidence.

Self-check: Name the captured testbed and deduplication changes. Separate training, validation, known test and novel test. State that replay order and timely feedback are experimental assumptions.

Source files: research/prepare.py, assets/research/dataset.json, assets/research/RESEARCH_REPORT.md

4 How one flow becomes a score

Twelve numbers enter trained equations; the returned number is an uncalibrated attack score.

গল্প and motivation

গল্প / imagined scenario: একই packet count দুই device-তে একই risk নয়; duration, packet rate এবং payload pattern একসঙ্গে matter করে। A trained model combines these measurements rather than assigning a fixed number of points to each field.

সহজ Analogy

সহজ Analogy: 12টি observation-কে 24টি ছোট pattern detector দেখে, তারপর একটি final score তৈরি হয়।

Analogy-র সীমা: hidden units have learned weights; they are not named rules like 'three SYN packets always mean an attack'.

Formal definition and code flow

transform() rejects vectors with wrong length or invalid numbers. For each feature it takes $\log(1 + \text{raw value})$, subtracts that feature's training mean, divides by its training standard deviation, then clips to -8 through 8 . The raw measurements retain their own units; transformed values are dimensionless.

forward() calculates 24 weighted sums plus biases, applies tanh, combines hidden outputs with another set of weights, and applies a numerically stable sigmoid. predict() returns the attack score p . The live display shows $100 \times (1 - p)$ as trust. $p \geq 0.5$ means FLAG_NETWORK_RISK, not deny access. These scores have not been calibrated to incident probabilities.

explain() sets one transformed feature at a time to zero, meaning the training mean in log space, and recomputes the score. Contribution = original attack score minus the altered score. Positive means the observed value raises the score relative to that background; negative means it lowers it. Contributions are perturbation comparisons, not SHAP, probabilities, causal effects or terms that must add to the prediction.

The 12 fields are duration, forward/backward packet totals, packet rate, down/up ratio, SYN/RST/ACK counts, payload mean/std, and forward/backward initial window sizes. Field names and units must match the published collector semantics. A browser cannot obtain arbitrary network traffic merely by opening this page.

Math and worked application

20. Teaching arithmetic only: attack score $p = 0.80$. Trust fraction = $1 - 0.80 = 0.20$. Displayed trust = $0.20 \times 100 = 20/100$. The threshold flags network risk.
21. Teaching attribution only: original attack score 0.80 ; replacing one transformed feature yields 0.55 . Contribution = $0.80 - 0.55 = +0.25$. This compares two inputs under the same model, not an intervention on the network.
22. The interactive activity below embeds the actual published initial weights. Choose an actual held-out example, then edit a measurement. Edited combinations may be physically inconsistent or outside training support; they are sensitivity probes, not newly observed flows.

Interactive check

Before changing packet rate, predict whether this fixed trained model's score must rise. Then test it; nonlinear models need not be monotonic.

Choose a real example and change packet rate. Compute the actual frozen MLP locally, show score/flag and edited-input caveat. Reset restores the original row. Zero and large rates are boundary probes, not new measurements.

Recall question: Does a trust score of 80 mean the user has an 80% chance of being authorized?

Answer: No; it is 100 times one minus an uncalibrated flow attack score. The inputs describe traffic, not identities or permissions. No calibration or authorization policy supports that interpretation.

Explain back: Trace raw flow measurements through preprocessing, the MLP and the displayed trust score.

Self-check: Explain log transform and training-only normalization. Identify the 12–24–1 MLP and score threshold. Explain one signed contribution without claiming causality.

Source files: runtime/engine.mjs, runtime/worker.mjs, assets/research/model.json, lab.mjs

5 When the stream suggests drift

An attack flag concerns one flow; a drift signal concerns changing model behavior over evidence.

গল্প and motivation

গল্প / imagined scenario: একটি unusual flow হতে পারে attack, কিন্তু তা দেখেই classifier বদলানো ঝুঁকিপূর্ণ। A stream may instead contain a sustained unfamiliar pattern. The detector asks whether enough evidence suggests the current model is struggling or relying on features differently.

সহজ Analogy

সহজ Analogy: একজন শিক্ষক একটি answer ভুল check করলে training বদলান না। Recent answers-এ ভুল বেড়ে গেলে বা decision pattern বদলে গেলে review করেন। Detection asks for review; it does not prove why a change happened.

Formal definition and code flow

processBatch() first predicts every row using the same current model. It then incorporates supplied labels into an error history and a recent labelled buffer. The error detector needs 256 observed labels: compare the last 64 errors against the preceding 192. Trigger when the short error fraction is strictly greater than the reference plus 0.08. This is a fixed-window heuristic, not a full ADWIN implementation.

The second signal averages absolute perturbation contributions over the current batch, normalizes their total, and compares the largest component change to a stored reference. It triggers when that maximum difference is strictly above 0.15. The initial reference comes from validation anchors; after a candidate attempt it is refreshed using recent labelled records under the active model.

A candidate additionally needs at least 128 seen flows since the last attempt, at least 64 labelled recent records, and some label evidence in the current batch. A score-only stream cannot invent labels to meet those requirements. After an attempt, error history is cleared, spacing restarts, and the reference is refreshed even if the candidate was rejected. The counters and windows therefore do not simply accumulate forever.

No candidate does not mean 'no attacks'. It may mean insufficient labels, no threshold crossing, insufficient spacing, or that only /api/infer was used. The current browser exposes stage indicators and candidate records, but does not present a continuous numerical detector panel for every non-triggering batch.

Math and worked application

23. Teaching counts: 12 errors in the recent 64 labels gives $12 / 64 = 0.1875$.

24. Teaching reference: 12 errors in the previous 192 gives $12 / 192 = 0.0625$.

25. $0.0625 + 0.08 = 0.1425$; $0.1875 > 0.1425$, so error drift is true once 256 labels exist.

26. If the gap is only 96 flows, the candidate is still blocked. Increase the gap to 128, retain 64 labelled recent records and current-batch feedback: an update attempt becomes eligible.

Interactive check

Predict whether the detector triggers and whether training is permitted; these are two different answers.

Change recent/reference error counts, attribution difference, available labels and spacing. Show signal and candidate eligibility separately using the actual thresholds. Reset to the worked case; equality does not pass strict drift thresholds.

Recall question: Attribution shift is 0.20 but the batch has no labels. Must the model adapt?

Answer: No, this implementation requires current label evidence Attribution can change on unlabelled inputs, but processBatch refuses a candidate without current label evidence and enough recent labels.

Explain back: Explain an observed zero-candidate count without assuming the service is broken.

Self-check: Distinguish flow classification from drift detection. Mention evidence, threshold and spacing requirements. Mention window resets after candidate attempts.

Source files: runtime/engine.mjs, assets/research/model.json, drift/detector.py

6 How a candidate learns

Training changes a copy first; the current batch is never rescored as if it had known its own labels.

গল্প and motivation

গল্প / imagined scenario: exam answers দেখে answer key মুখস্থ করার পর সেই একই attempt-কে 'আগেই জানতাম' বলা unfair। The evaluation avoids that ordering error by storing predictions before it exposes the batch's labels.

সহজ Analogy

সহজ Analogy: software patch staging—active program চলতে থাকে, copied version-এ change তৈরি হয়, তারপর checks run করে deploy decision নেওয়া হয়। Replay adds revision exercises so a candidate is not trained only on the latest examples.

Formal definition and code flow

The Worker divides a replay or CSV request into batches of at most 32 and calls processBatch() sequentially. Each batch's predictions are made first. Its labels then update evidence and can help train a candidate that affects subsequent batches. A 128-flow API request contains four prediction-then-feedback cycles, not one model frozen across all 128 flows.

When eligible, the engine structured-clones the active model. fit() runs three SGD passes at learning rate 0.003 with a 0.0001 weight penalty. The ordinary adaptation modes use up to 128 recent labelled records plus up to 128 replay records. The no_replay ablation trains only on recent records. Initial sklearn Adam fitting and online JavaScript SGD are different stages.

For each row, forward propagation supplies hidden activations and p. The output error is $p - y$. Gradients are calculated, then output weights, hidden biases/input weights and output bias are adjusted. The resulting candidate is checked on identical validation anchors before any activation. This is real incremental weight training; no complete retraining is scheduled on every button click.

After an attempt the replay buffer appends recent records and keeps its last 128 entries. This is a rolling replay mechanism, not a guaranteed permanent memory of all old classes. Recent records can overlap with replay records on later attempts, so '256 training entries' does not necessarily mean 256 distinct flows.

Math and worked application

27. Teaching one-weight update, not the complete real network: $y = 1$, $p = 0.20$, hidden activation = 0.5, old output weight = 0.4, learning rate = 0.003.

28. Prediction error = $0.20 - 1 = -0.80$. Data-gradient contribution = $-0.80 \times 0.5 = -0.40$.

29. Weight penalty = $0.0001 \times 0.4 = 0.00004$. Total gradient = -0.39996 .

30. Change = $0.003 \times -0.39996 = -0.00119988$. New weight = $0.4 - (-0.00119988) = 0.40119988$. The other weights also change in the real engine.

Interactive check

Arrange prediction, feedback, candidate training, policy checks and activation in their valid order. Predict what is wrong with revealing labels first.

Use selects to place prediction, feedback, training, checking and activation in order. Show the first order violation and why it matters. Reset scrambles the steps; no retroactive prediction rewriting is allowed.

Recall question: Which predictions can a candidate trained after batch 1 affect?

Answer: Later batches after acceptance The batch's scores are saved before labels enter learning. Accepted weights are used for later calls; previous scores are not rewritten.

Explain back: Explain why offline and live results use a test-then-train sequence.

Self-check: Predict the whole 32-flow batch before feedback. Train a candidate copy using recent and replay entries. Apply accepted weights only to subsequent predictions.

Source files: runtime/engine.mjs, runtime/worker.mjs, research/run.mjs, tests/engine.test.mjs

7 Why a candidate is accepted or rejected

Audit-only documents checks; gated mode makes those same checks veto an update.

গল্প and motivation

গল্প / imagined scenario: নতুন attack চিনতে শেখা patch পুরোনো traffic-এ ভুল বাড়তে পারে। A fixed test set can detect that regression, but it can also prefer old behavior so strongly that useful new learning is blocked.

সহজ Analogy

সহজ Analogy: audit-only হলো inspection report সংরক্ষণ করে patch চালু করা; gated হলো mandatory inspection fail করলে patch আটকে রাখা। Passing a small checklist is not proof that every real-world requirement is satisfied.

Formal definition and code flow

The candidate and active model are evaluated on the identical fixed validation-anchor rows. Checks require ECI at least 0.5, anchor log-loss increase no more than 0.05, attack-anchor miss-rate increase no more than 0.05, and at least 128-flow spacing. The last condition is mostly enforced before training already. The policy mapping to ISO controls is a research proxy, not a certification assessment.

ECI compares anchor-based importance rankings before and after training. Top-5 Jaccard measures intersection divided by union; normalized Kendall tau-b measures ranking agreement with ties. ECI averages Jaccard and $(\text{tau} + 1) / 2$. A high ECI means explanations remained similar by these measures; it does not prove explanation correctness or fairness.

DriftTrust Audit Code and visitor guide

Audit-only (monitor_only) accepts candidates, retaining failed_checks and status ACCEPTED_WITH_FLAGS when needed. Gated (governed) activates a candidate only if all checks pass; otherwise status is REJECTED and the old model hash remains active. A rejected model candidate is not a blocked network request. Ungoverned, static, no_replay and no_attribution are offline comparison modes, not additional choices in the live dropdown.

Measured shifted F1 across five seeds: frozen 0.4060 ± 0.1239 ; audit-only and ungoverned 0.9064 ± 0.0191 ; gated 0.5872 ± 0.1600 . The adaptive pair has identical predictions by design; logging does not itself improve classification. Audit-only also has a higher benign false-alarm rate than the gate in this experiment. Gate sensitivity and anchor design remain open research work.

Math and worked application

31. Teaching ECI: two top-five sets share four features, so their union has six. Jaccard = $4 / 6 = 0.6667$.
32. Teaching Kendall tau-b = 0.60. Normalize: $(0.60 + 1) / 2 = 0.80$. ECI = $(0.6667 + 0.80) / 2 = 0.7333$.
33. Teaching candidate: loss rises from 0.20 to 0.28, an increase of 0.08. ECI passes but the 0.05 loss limit fails.
34. Audit-only accepts this candidate with flags. Gated rejects it and keeps old weights. The calculation illustrates the policy; it is not an observed dataset event.

Interactive check

Choose the mode, then vary ECI and anchor changes. Predict which single failure is enough to block a gated candidate.

Change ECI, loss/miss increases and policy mode. Show failed checks, activation/rejection and active model identity. Reset to the failed-loss example; equality passes gate limits.

Recall question: Does ACCEPTED_WITH_FLAGS mean every governance check passed?

Answer: No; audit-only accepted despite failed checks The status refers to activating a model candidate under audit-only policy. failed_checks records what did not pass.

Explain back: Explain the measured benefit and cost of gating without claiming it is always superior.

Self-check: Distinguish candidate acceptance from flow permission. Explain identical-anchor ECI and performance checks. Report lower shifted F1 for the current gate and its false-alarm tradeoff.

Source files: runtime/engine.mjs, assets/research/RESEARCH_REPORT.md, assets/research/results.json

8 Audit records and session persistence

The evidence describes actual candidate attempts and links each record to the previous one.

গল্প and motivation

গল্প / imagined scenario: model update-এর পরে reviewer জিজ্ঞেস করলেন 'কী শিখল, কোন records ব্যবহার করল, এবং কেন চালু হল?' শুধু final accuracy দিয়ে এই questions-এর উত্তর দেওয়া যায় না। AJR preserves the candidate decision context.

সহজ Analogy

সহজ Analogy: প্রতিটি page-এ আগের page-এর fingerprint রেখে notebook লিখলে মাঝের page বদলানো detectable হয়। কিন্তু কেউ পুরো notebook এবং শেষ fingerprint দুটোই নতুন করে লিখতে পারলে বাইরে রাখা trusted copy ছাড়া ধরা যায় না।

Formal definition and code flow

An AJR includes attempt and stream position, time, error/attribution trigger evidence, exact training and anchor IDs, model-before/candidate/active SHA-256 hashes, same-anchor losses and misses, ECI components, check booleans, failed checks, status, feedback source and incremental losses. It records each attempted adaptation, including rejection; it is not a complete log of all individual predictions.

canonical() sorts object keys for stable serialization; digest() hashes canonical JSON. Each record stores previous_record_sha256 and its own record_sha256; state stores chain_head. The verifier recomputes every hash and checks the links and head. Hashes identify content; they do not prove the labels were true, prevent all malicious activity, or let you reconstruct model weights from the hash alone.

The Worker creates an unguessable session cookie, stores only its hash as the session key, and reads that session's model before operations. D1 tables hold sessions, audit records and creation-limit buckets. The model state includes recent/replay learning buffers and the last response. There is no global learning from all visitors; one person's labels affect only that person's session.

Requests use revisions to prevent silent concurrent overwrites. Database batches persist state and candidate records together. The most recent request_id can be retried for the saved response; stale revisions return 409. Sessions expire after 24 hours, with later session creation cleaning expired records. The service limits 20 creations per hour per IP bucket and 20,000 flows per session. Reload can resume a valid session via cookie, but browser-only chart history is not restored as a complete historical trace.

Math and worked application

35. Record A has hash hA. Record B includes previous_record_sha256 = hA before its own hash hB is calculated.
36. Change B's status without changing its saved hash: recomputed digest no longer equals hB, so verification fails.
37. Rewrite both records and all their hashes: an exported head trusted elsewhere is needed to detect substitution of the whole chain.
38. Download session AJRs and run node scripts/verify-audit.mjs downloaded-session-audit.json. A successful check proves chain consistency relative to the provided head, not truth of every claim.

Interactive check

Verify a copied real experiment record, then change its status and predict whether the stored fingerprint still matches.

Verify a copied real primary-run AJR using canonical JSON SHA-256; edit status and reverify. Show mismatch and explain trusted-head limits. Reset restores the untouched copy; no original records change.

Recall question: Is an AJR created for every single-flow inference?

Answer: No; it records adaptation candidate attempts /api/infer does not train. AJRs are built inside processBatch when a candidate is trained and evaluated.

Explain back: Describe what an audit export proves and what it cannot prove.

Self-check: Mention exact evidence and model fingerprints. Separate hash-chain integrity from truth and immutability. Explain visitor isolation and limited session retention.

Source files: runtime/engine.mjs, runtime/worker.mjs, db/schema.ts, scripts/verify-audit.mjs, assets/research/DEPLOYMENT.md

9 Reading evidence and defending the project

A working research system is valuable when its measured behavior and remaining scope are clear.

গল্প and motivation

গল্প / imagined reviewer conversation: ‘Your demo runs, but does it establish general enterprise security?’
উত্তর দিতে implementation evidence, dataset findings এবং proposed integrations আলাদা করতে হবে। Strong explanations survive both a successful example and a known mistake.

সহজ Analogy

সহজ Analogy: test track-এ গাড়ি চালানো real engineering evidence; এটি সব রাস্তা ও আবহাওয়ায় accident-free driving-এর proof নয়। Likewise, measured IoT flows support a defined benchmark and live research experience.

Formal definition and code flow

research/report.py calculates attack-positive classification metrics, per-family behavior, rolling diagnostics and governance figures from stored predictions. Nine figures have PDF/SVG/PNG exports. Aggregate plots use five splits; detailed plots use seed 11. The online 1,280-flow compact replay is not the complete 7,972-flow shifted paper test.

Attack F1 combines attack precision and recall; it is not overall accuracy, calibrated probability or percent of all flows correct. The reported standard deviation describes variability over five overlapping dataset splits. The splits are not five independent real-world deployments. Stationary random forest F1 is strongest among these configurations; the adaptation advantage is conditional on the constructed shifted protocol.

tests/engine.test.mjs checks independent sklearn prediction parity, input rejection, ECI properties, prediction-before-feedback, no-label learning behavior, audit-only parity and rejected-update integrity. tests/api-check.mjs checks actual API behavior, isolation, retries, replay and chain consistency. The recorded hosted check processed 1,280 flows with five accepted audit-only candidates and maximum local/cloud replay-score difference about $8.33e-17$. These are behavioral checks, not penetration testing, enterprise certification or a complete browser-usability study.

For real deployment beyond this lab, an authorized flow collector must produce compatible measurements; trusted feedback must be connected; score calibration and delayed-label robustness need evaluation; identity and policy-enforcement integrations must be implemented if making access-control claims. The current UI exposes a real flow-risk model, not those unimplemented components. No tool can guarantee that reviewers accept the research paper.

Math and worked application

39. Teaching metric counts: true attacks correctly flagged = 80, benign flows wrongly flagged = 20, attacks missed = 40.
40. Precision = $80 / (80 + 20) = 0.80$. Recall = $80 / (80 + 40) = 0.6667$.
41. $F1 = 2 \times 80 / (2 \times 80 + 20 + 40) = 160 / 220 = 0.7273$. This is not an assertion that 72.73% of every type of flow was classified correctly.
42. Suggested presentation: 'We trained on a public measured IoT dataset, replayed held-out flows with explicit feedback, and deployed the same inference/update engine. Visitors can inspect real predictions and model-change records. Gating reduced some errors but limited useful adaptation under this protocol.'

Interactive check

Change false positives and missed attacks while keeping correct detections fixed; predict which metric responds and what a visitor should infer.

Adjust TP/FP/FN counts and compute precision, recall and F1. Show each denominator and distinguish F1 from accuracy. Reset to 80/20/40; empty denominators return zero by a stated teaching convention.

Recall question: Which claim is supported by the present evidence?

Answer: Working flow-risk inference and auditable adaptation on a stated testbed protocol The source, measured runs and hosted checks support that bounded claim. Broader enforcement, certification and population-generalization claims require further evidence.

Explain back: Give a 60-second project explanation including one strength, one measured tradeoff and one unimplemented integration.

Self-check: Identify public measured data and the shared deployed engine. Explain inference → feedback → candidate → audit/governance. State the gate tradeoff and enterprise-enforcement limitation.

Source files: research/report.py, tests/engine.test.mjs, tests/api-check.mjs, assets/research/deployment_verification.json, assets/research/RESEARCH_REPORT.md

Repository map

File or directory	Responsibility
main.py; config.py	Legacy synthetic orchestration and settings; not the live API entry point.
data/simulate.py; data/features.py	Generate synthetic telemetry and window/normalize it for the legacy model.
models/trust_scorer.py; models/trainer.py	Legacy flattened-window sklearn MLP wrapper, initial fitting and persistence.
drift/detector.py; attribution.py; adapter.py	Legacy error/perturbation detection and continual replay adaptation.
governance/eci.py; policy.py; ajr.py	Legacy explanation consistency, policy proxies and JSON records.

File or directory	Responsibility
evaluation/	Legacy baseline scores, metrics, governance summaries and charts.
research/prepare.py	Pinned dataset download, audit, split, train, export weights/streams/baselines.
research/run.mjs	Five seeds × two protocols × six engine modes, plus two static baselines.
research/report.py	Numerical summaries, nine scientific figures and result archives.
runtime/engine.mjs	Preprocessing, MLP, attribution, error evidence, candidate SGD, ECI and chained AJRs.
runtime/worker.mjs	HTTP routing, input/session validation, engine calls and D1 transactions.
db/schema.ts; drizzle/	Database table definitions and deployment migrations.
index.html; site.mjs; site.css	Public research page, real results table, figure links and visual styles.
lab.html; lab.mjs	Live input forms, session controls, replay/upload actions and response rendering.
assets/research/	Published model, examples, replay, audit evidence, tables, figures and documentation.
scripts/build.mjs; scripts/dev.mjs	Bundle Worker/static site; local server with SQLite D1 adapter.
scripts/package-site.ps1; .openai/hosting.json	Windows packaging and Sites project/database binding metadata.
scripts/verify-audit.mjs; tests/	Hash-chain verification, scientific engine checks and API integration checks.
package.json; pnpm-lock.yaml; requirements-research.lock.txt	Commands and locked Node/Python dependencies for reproducibility.
.github/workflows/; informations/	Automated checks and historical methodology/status documents. Historical claims are not substitutes for the current report.

HTTP request map

Route	Purpose
GET /api/health	Model and database availability
GET /api/model	Feature schema, examples, published configuration
POST /api/session	New audit-only or gated session

Route	Purpose
GET /api/session	Current counters, mode, revision, cursor
POST /api/infer	One x vector; read-only prediction and explanation
POST /api/replay	revision, request_id, count; server-held records
POST /api/stream	revision, request_id, rows with x and optional y
GET /api/audit	Session candidate records and chain head

Developer commands and safe interpretation

`pnpm install --frozen-lockfile` → install the Node dependency lock. `pnpm dev` → local lab at `http://localhost:8787/lab.html` with SQLite-backed session state.

`python research/prepare.py` → retrieve/check data and fit initial models; `node research/run.mjs` → execute streaming comparisons; `python research/report.py` → regenerate metrics and figures. Install `requirements-research.lock.txt` in Python 3.12 first.

`pnpm test` → scientific engine tests. With the local server running, `node tests/api-check.mjs` `http://localhost:8787` → API behavior checks. `pnpm build` → static assets and Worker bundle. Build does not itself publish.

`node scripts/verify-audit.mjs downloaded-session-audit.json` → recompute canonical hashes and verify chain links. A trusted saved head strengthens later substitution detection.

The original `main.py` is a separate synthetic workflow and clears files in its configured audit/results output directories at startup. Do not confuse running it with starting the live service.

Troubleshooting the visitor experience

Service unavailable: open the deployed lab or run `pnpm dev`; double-clicking `lab.html` alone cannot run its Worker API. The offline fieldbook is intentionally different and works without a server.

No candidates: ensure you used replay or a labelled stream, supplied enough labels and met spacing/trigger rules. One suspicious flow does not force retraining.

Policy appears unchanged: select a policy and then start a new session. A selection alone does not mutate the existing session.

Unexpected score: compare known truth separately, inspect training-range warnings and remember scores are uncalibrated. Edited example values no longer have a validated dataset label.

CSV rejected: check exact header names, 12 numeric features, non-negative finite values, 0/1/blank labels, maximum 256 rows and 2 MB browser file size. API JSON bodies have a separate 400 KB limit.

Session expired or conflict: create a new session after expiry; a 409 requires refreshing current revision. Export useful evidence before replacing the session. The last-request retry protection is not an unlimited request-history cache.

Glossary

Flow — একই bidirectional network conversation-এর measurements; not a login identity.

Inference — বর্তমান weights দিয়ে নতুন input-এর score হিসাব করা.

Label — Feedback: current backend uses 0 benign, 1 attack, null unknown.

Drift — Changing error or attribution behavior across evidence; not synonymous with one attack.

Candidate — A trained model copy awaiting activation policy.

Anchor — Fixed validation record used to compare old and candidate models fairly.

Replay buffer — Older labelled entries mixed into an incremental update; bounded rolling memory.

ECI — Explanation Consistency Index, based on ranking similarity across identical anchors.

AJR — Adaptation Justification Record, evidence about one attempted model update.

D1 — The deployed database holding isolated session model state and audit records.

Revision — Session version number used to detect conflicting writes.

F1 — A score combining positive-class precision and recall; not overall accuracy.

False negative — An attack that receives the lower-risk classification.

False positive — A benign flow that receives the network-risk flag.

Calibration — Whether score values correspond reliably to observed event frequencies; not established here.

Hash chain — Content fingerprints linked across records; useful with a separately trusted head.

Sources and claim boundaries

Repository at the inspected snapshot: https://github.com/the-sudipta/drifttrust_audit/tree/32e224d901843acc4761f25fd02d93f73a5174f0. Module source-file locators identify the implementation behind each explanation.

Dataset: B. S. Sharmila and Rohini Nagapadma. RT-IoT2022. UCI Machine Learning Repository. DOI 10.24432/C5P338. CC BY 4.0. <https://archive.ics.uci.edu/dataset/942/rt-iot2022>

Measured findings: assets/research/RESEARCH_REPORT.md, dataset.json, results.json and deployment_verification.json. This dossier adds explanation, not a new benchmark run. Actual initial-model example outputs were recomputed directly from the published checkpoint for this guide.

Teaching scenarios and policy/metric arithmetic are explicitly illustrative. The embedded score activity uses real frozen weights; editing a feature creates a sensitivity probe. The offline guide never modifies a hosted model, submits measurements or creates live AJRs.